Some of the problems below ask you to convert between characters and their ASCII values. Here's a table to help.

Char	Binary	Char	Binary
a	01100001	n	01101110
b	01100010	o	01101111
c	01100011	p	01110000
d	01100100	q	01110001
e	01100101	r	01110010
f	01100110	s	01110011
g	01100111	t	01110100
h	01101000	u	01110101
i	01101001	v	01110110
j	01101010	w	01110111
k	01101011	x	01111000
l	01101100	y	01111001
m	01101101	z	01111010

Table 1. Lowercase ASCII Binary Mapping

1. The OTP encryption of the string `hi` is `1100110011000111`0. What is the encryption of the string `um` using the same OTP key?

2. Your colleague has written some software to perform OTP encryption. It matches what we saw in class, with one exception: instead of XORing the plaintext and key, it performs unsigned 64-bit addition. (That is, the binary strings are interpreted as positive binary integers, and the *Enc* algorithm adds the plaintext and key together, modulo 2^{64}.)

   (a) What should your colleague's decryption algorithm be to satisfy the correctness property of OTP encryption?

   (b) Does your colleague's scheme satisfy the security property of OTP encryption?

   (c) Another colleague comes along and suggests you use unsigned 64-bit multiplication instead of addition. Is this a valid encryption scheme?